

International University

School of Computer Science and Engineering

Web Application Development

Laboratory

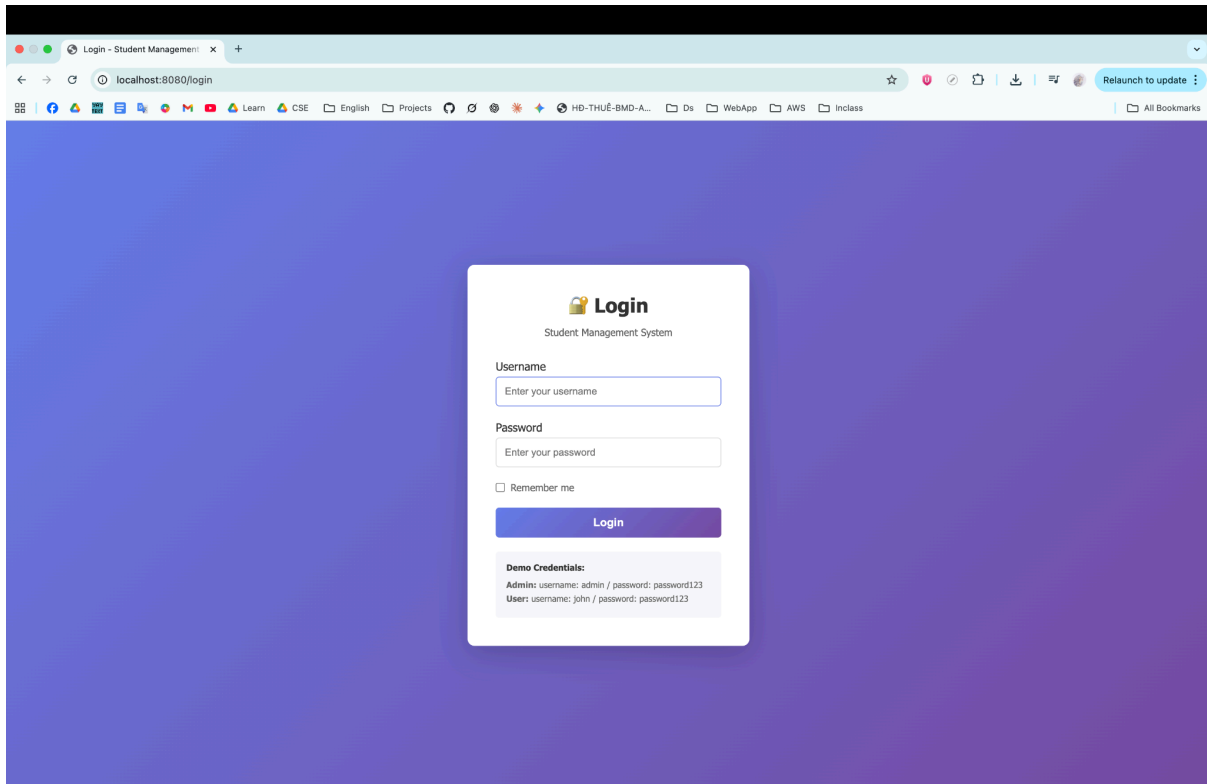
IT093IU

Lab #6 Practice

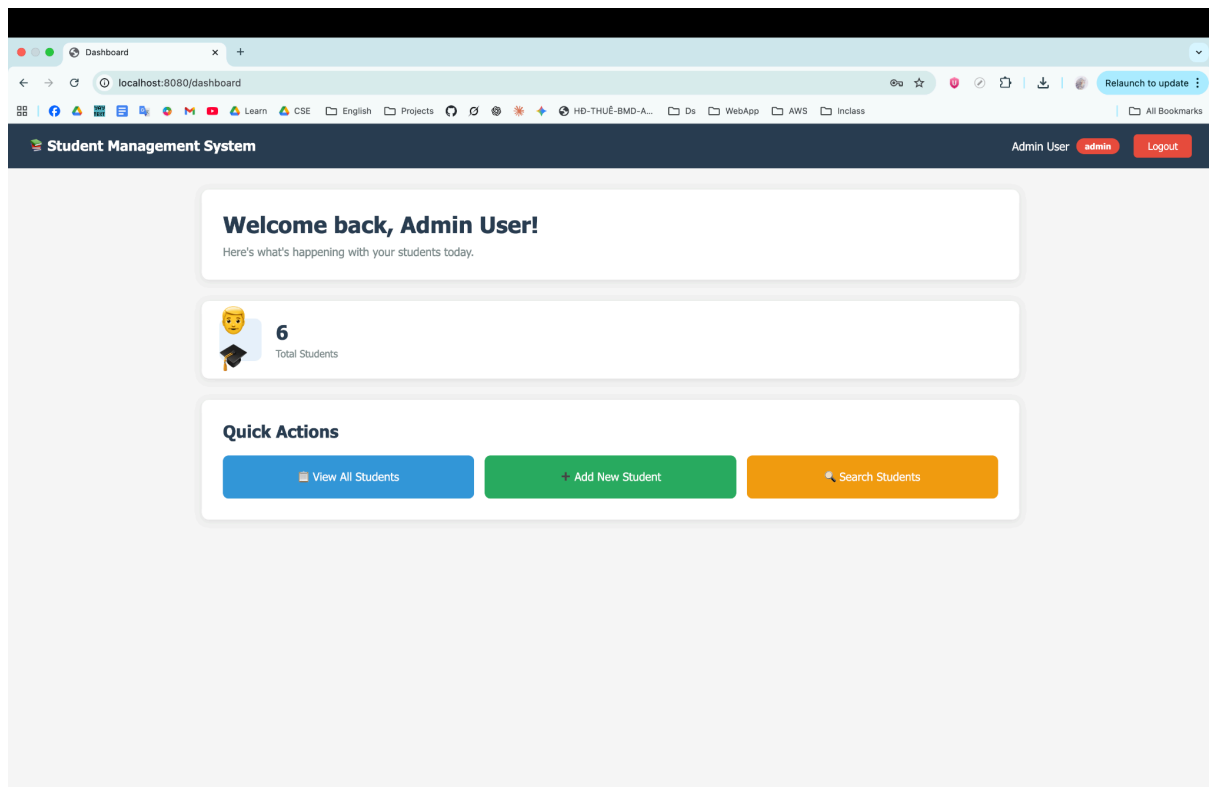
Submitted by

Nguyễn Hồng Ngọc Hân - ITCSIU22229

Test login flow



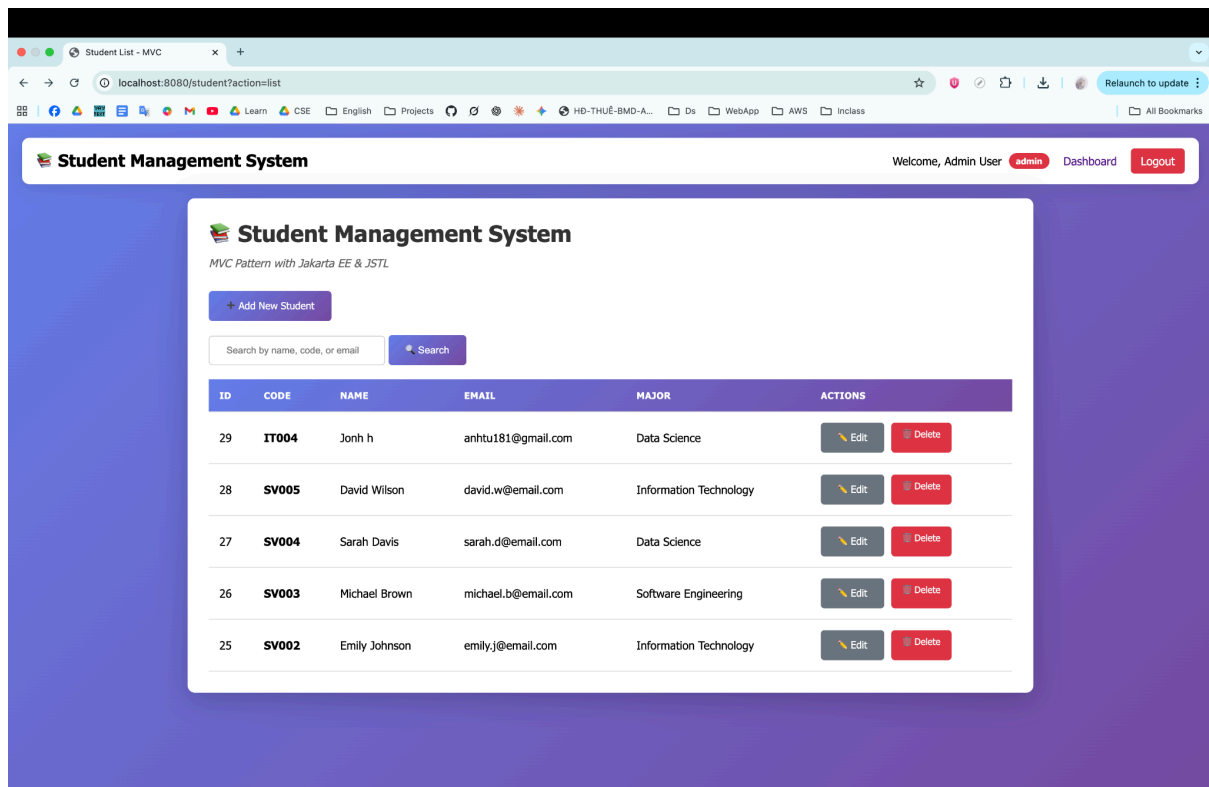
When I try to open the student list page, I am redirected to the login page. This happens because every request is checked by an authentication filter called `AuthFilter`. The filter examines the requested URL to determine whether it belongs to the “public” routes, such as `/login`, `/logout`, or static resources like CSS, JavaScript, or images. If the request is for a public resource, it is allowed to continue. For any other protected page, the filter verifies whether the user is already logged in by checking the session for a stored user object. If the session contains this user attribute, the request is allowed. If not, the filter prevents access and sends the user to the login page. In summary, the `AuthFilter` ensures that only authenticated users can access the student management features.



When a user goes to `/login`, the `LoginController` first checks whether the user is already logged in by looking for `session.getAttribute("user")`. If this attribute exists, the user is immediately redirected to the dashboard. If not, the controller loads and displays `login.jsp`.

When the login form is submitted (via a POST request), the controller reads the username and password, makes sure both fields are filled in, and then calls `userDAO.authenticate()` to verify the credentials. If the authentication is successful, the controller first invalidates any previous session, creates a new one, and saves the logged-in user's details in session attributes such as `"user"`, `"role"`, and `"fullName"`. These values are later used by the `AuthFilter` and by JSP pages. The session is also configured to expire after 30 minutes of inactivity. Depending on the user's role, the controller redirects them either to `/dashboard` (for admins) or to `/student?action=list` (for regular users).

If authentication fails, the login page is shown again with an error message. The `login.jsp` file simply renders the login form and displays any message passed from the controller, such as an error or a success notice. The form then posts the entered credentials back to `/login` for processing.

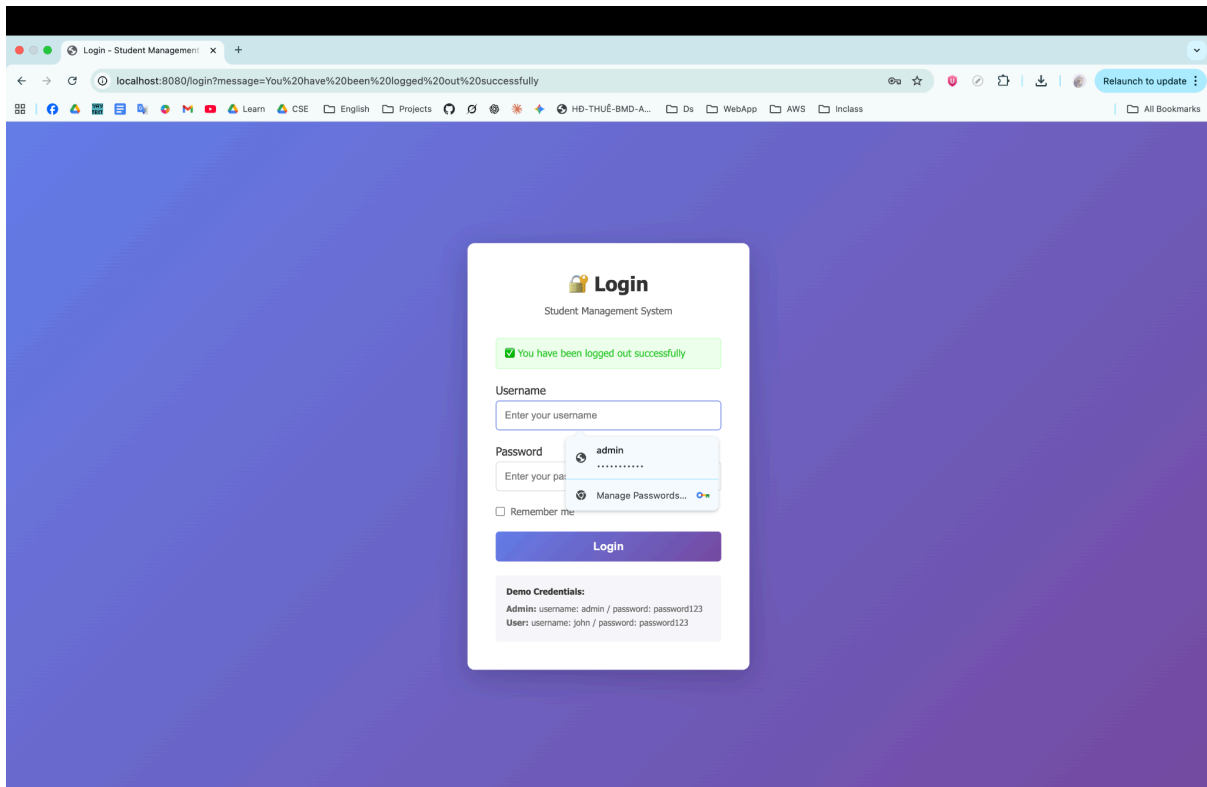


When the user clicks “View All Students”, the page loads only if the user is already authenticated. This is enforced by the `AuthFilter`, which checks the session for attributes like `fullName` and `role`. Once access is granted, the top navigation bar shows the logged-in user's information using `sessionScope.fullName` and `sessionScope.role`.

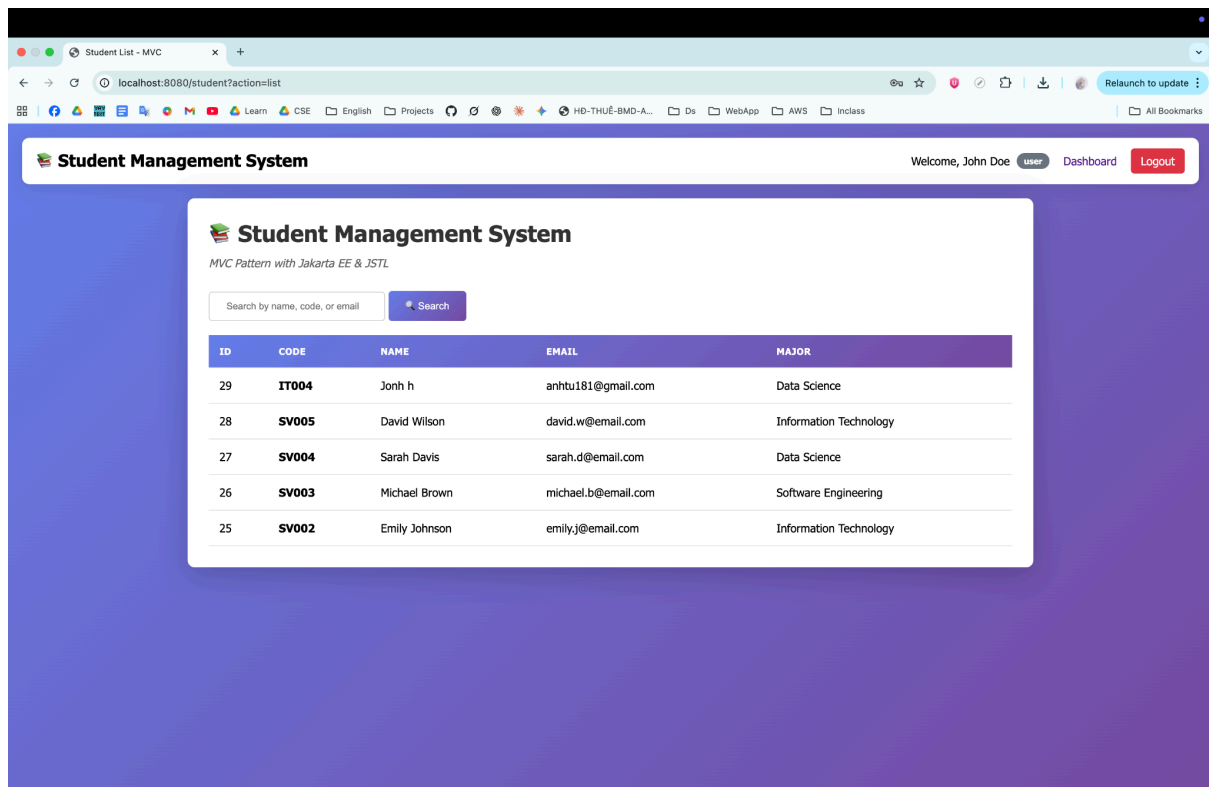
If the user has the `admin` role, the JSP enables additional admin-only features such as the `Add New Student` button and the `Edit/Delete` actions. These are displayed conditionally using `<c:if>` checks that verify whether the session role equals `"admin"`.

The main table displays all student records sent from the controller through the `${students}` attribute. If no data is available, the page shows a friendly “No students found” message. Pagination elements appear only when multiple pages exist, using `${totalPages}` and `${currentPage}`—along with JSTL loops—to build the numbered page links.

In summary, this JSP page renders the complete student list interface, personalizes the header using session data, and shows/hides admin-only controls depending on the user's role.



When the user goes to /logout, the controller first looks for an existing session. If a session is present, it invalidates it—removing all stored information such as the user object, fullName, and role—effectively logging the user out of the system. After clearing the session, the servlet redirects the user back to the login page and includes a success message in the URL (login?message=You have been logged out successfully) so the login screen can display a confirmation notice. If the logout request is sent using POST, the doPost() method simply calls doGet(), ensuring that the logout process behaves the same regardless of whether the request is GET or POST.



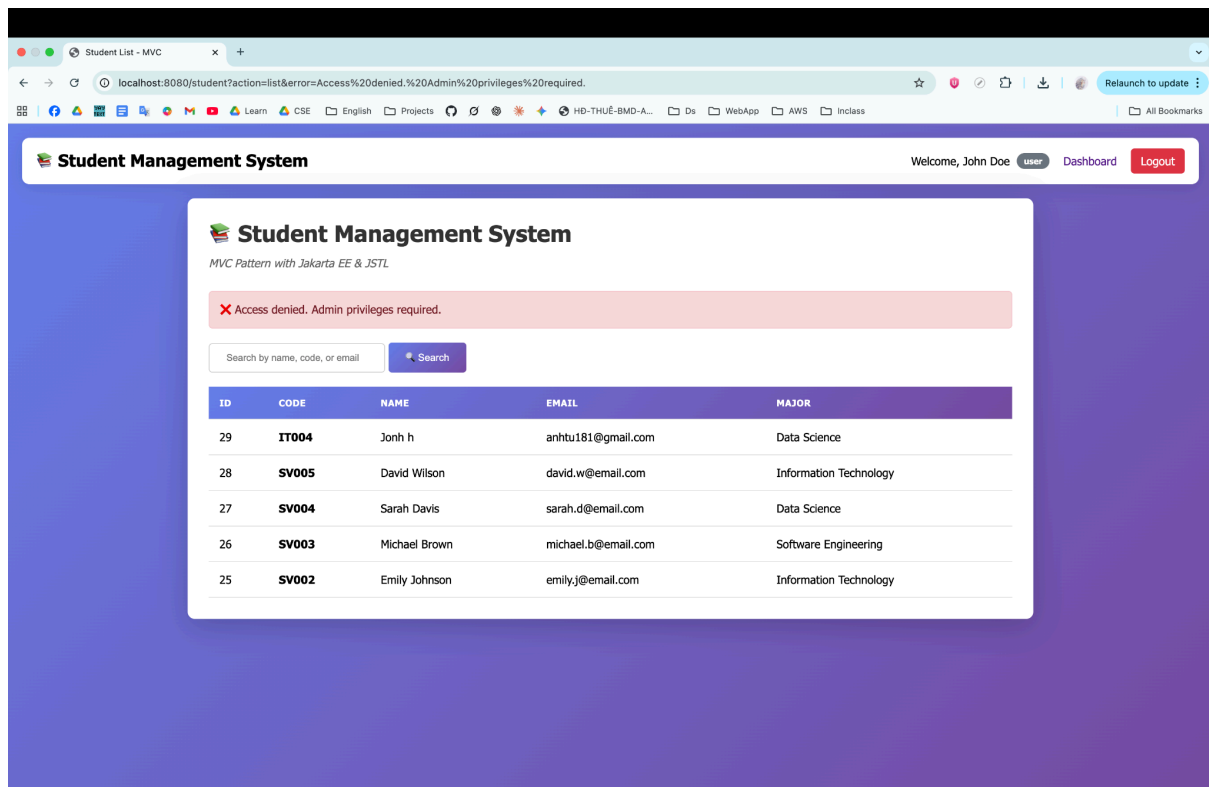
Login with User Account (john / password123)

When the user navigates to `/login`, the `LoginController` first checks whether the user is already authenticated by looking for `session.getAttribute("user")`. If the attribute exists, the user is immediately redirected to the dashboard. If not, the controller displays `login.jsp`, which contains the login form.

When the user submits the form through a POST request, the controller reads the entered username and password, verifies that both fields are filled in, and then calls `userDAO.authenticate()` to validate the credentials. If the login is successful, the controller clears any previous session, creates a fresh one, and saves the logged-in user's details in session attributes such as `"user"`, `"role"`, and `"fullName"`. These session values are later used by the `AuthFilter` and JSP pages to control access and personalize the UI. The session timeout is also set to 30 minutes for security.

After authentication, the user is redirected either to `/dashboard` (for admins) or to `/student?action=list` (for regular users). If the credentials are invalid, the login page is reloaded with an error message.

The `login.jsp` page simply renders the login form and displays any messages sent by the controller, such as errors or a logout success notice. When submitted, the form posts the credentials back to `/login` for processing.



The AdminFilter intercepts every request targeting the /student URL before it reaches the servlet. When a request such as /student?action=edit arrives, the filter reads the action parameter and checks whether it belongs to the list of admin-only actions: new, insert, edit, update, and delete. If the action is not restricted—for example, listing or searching students—the filter simply lets the request pass through.

For protected actions, the filter first checks whether the user is logged in by looking for an existing session. If no session is found, the user is redirected to the login page. If the session exists, the filter retrieves the user object saved during login and verifies the user's role. If the role is admin, the request is allowed to continue. If the user is not an admin, the filter blocks the request and redirects them back to the student list with an "Access denied. Admin privileges required." message.

In short, the AdminFilter ensures that only users with admin rights can create, edit, or delete student records, while regular users can still access safe, non-restricted pages.